

Chapter 03: The Perceptron: Learning a Decision Boundary

Table of Contents

[Chapter Introduction](#)

[The Step Function](#)

[Perceptron Training Algorithm](#)

[Perceptron Learning Rule](#)

[Why Multiply with \$x_i\$?](#)

[What a Perceptron Learns: A Closer Look](#)

[Limitation of a Perceptron](#)

[Perceptron and Logistic Regression](#)

Chapter Introduction

In Machine Learning, **the whole point is that a model learns its parameters (including weights, bias) from data.**

Then, **how can an artificial neuron learn (its parameters)?**

The first successful answer to this question was the **Perceptron**, proposed by Frank Rosenblatt in 1957.

It was the first algorithm that could automatically adjust its own weights based on mistakes. The algorithm also specifies an activation function, namely, the Step Function.

So, **Perceptron is an algorithm** that comes with an activation function, to build and train an AN.

Also, An AN that has been built and trained with the Perceptron Algorithm is called a Perceptron. Hence, **Perceptrons are a kind of Artificial Neurons.**

The Step Function

$$a_{\text{step}}(z) = \begin{cases} 1, & \text{if } z \geq 0 \\ 0, & \text{if } z < 0 \end{cases}$$

This is the activation function for a Perceptron. z is the Linear Weighted Sum.

This step function decides on just the sign of z . This function is not linear, but also barely non-linear. It's just a combination of two linear branches. Hence, its expressive power is no more than linear.

Its range is the discrete set $\{0, 1\}$ and the Perceptron Training Algorithm is supervised, so **a Perceptron is a Binary Classifier**.

Perceptron Training Algorithm

1. Initialize Weights and Bias randomly.
2. Iterate over observations in the training set (This iteration process usually is more than one epoch long). On each iteration—
 - a. For the current observation X , calculate the prediction $\hat{y} = a_{\text{step}}(z(X))$.
 - b. Update Weights and Bias, using **Perceptron Learning Rule**, as described in the next section.

Perceptron Learning Rule

$$w_{i,\text{new}} = w_{i,\text{old}} + \eta (y - \hat{y}) x_i$$
$$b_{\text{new}} = b_{\text{old}} + \eta (y - \hat{y})$$

The two equations above are collectively called the **Perceptron Learning Rule**, or the **Perceptron Trick**.

When a training example is predicted truly (either True Positive or True Negative) based on current Weights and Bias, then for this example, neither Weights nor Bias updates, as $y - \hat{y}$ becomes 0.

When a training example is predicted falsely (either False Positive or False Negative) based on current Weights and Bias, then for this example, at least Bias updates. The weights will not update when $x_i = 0$.

The rule is remarkably intuitive: **Reward correct direction. Correct mistakes by nudging the boundary.**

η is the learning rate, a hyperparameter of the Perceptron Model. It controls the step size of the update. A small Learning Rate is stable, but slow. A large Learning Rate is fast, but with risk of overshooting.

If η equal 1, then the current misclassified training example is guaranteed, by the algorithm, to be classified truly, after the update of Weights and Bias, in response to its misclassification, but might mess up other training examples by overshooting.

See [this Learning Rule in Action](#).

Why Multiply with x_i ?

If, for a misclassified training example, $x_i = 0$, then Δw_i should be 0, as the feature has no influence in this example, changing w_i is not only unnecessary, but also, according to the geometric mechanics of the training algorithm, is not helping at all. (Hint: Updating an Weight is a Rotation of the Decision Boundary, while Updating the Bias is a Translation/Shift of the Decision Boundary.)

Also, when $x_i \neq 0$, its sign and absolute value indicates the direction and amount of rotation, respectively, needed for the tentative decision boundary.

So, x_i must be in the Weight Update Formula.

What a Perceptron Learns: A Closer Look

It's obvious that a perceptron learns W and b , during training.

During prediction for an observation X , it uses W and b to calculate the Weighted Sum z .

The prediction is based on the sign of z .

If $z \geq 0$, the class prediction is 1.

If $z < 0$, the class prediction is 0.

Geometrically, with X as a feature variable, $z \geq 0$ and $z < 0$ are two regions in the feature space, separated by $z = 0$ hyperplane as a boundary between these two mutually exclusive regions. And the union of these two regions is the whole feature space.

For $z \geq 0$ region, class prediction/decision is 1, and for $z < 0$ region, class prediction/decision is 0. That is why the regions are called decision regions and the hyperplane $z = 0$ is called the linear Decision Boundary.

Hence, **the Perceptron does not learn classes. It learns a decision boundary.**

The Perceptron does not memorize classes—it learns a decision boundary. That means, it truly learns.

Limitation of a Perceptron

A Perceptron is only a Binary Classifier, and it can solve Binary Classification Problem, only on a Linearly Separable Binary Classification Dataset.

Perceptron and Logistic Regression

Both first computes the Weighted Sum on an input observation.

After that,

- The Perceptron, using the non-differentiable Step Function as its Activation Function, deterministically classifies the input observation.
- Logistic Regression, using the Sigmoid Function, probabilistically classifies the input observation.

Another key difference is **how they learn**.

- The Perceptron updates weights only when it misclassifies an example, using the Perceptron Learning Rule.
- Logistic Regression minimizes a differentiable loss function (cross-entropy) using gradient descent.

These two ideas are connectible with the concept of Backpropagation.